

Table of Contents

가시화 요구사항 — 쉽게 설명하기

2026-08-28 개정 — 「임무 설계 및 디버깅」이 탭⑥에서 탭①이 되었습니다. 이 앱에서 새로 만드는 유일한 탭이고 논문의 대상이라, 본류가 맨 끝에 있으면 앱을 처음 여는 사람이 무엇을 보는 도구인지 알 수 없습니다. 나머지 다섯이 한 칸씩 밀렸습니다 — 구역 현황판 ①→②, 제어 패널 ②→③, 지표 조회 ③→④, 영상 오버레이 ④→⑤, 파이프라인 편집기 ⑤→⑥. **요구사항 내용은 하나도 바뀌지 않았고 번호만 바뀝니다.**

2026-08-27 개정 (2차) — 사라진 계획 생성을 가시화가 인수하기로 정해졌습니다. AI 팀원이 논문 작업을 병행하게 되어 텍스트→서브태스크 생성을 제가 맡고, 검증·마일스톤 분리·평가 판정까지 함께 가져옵니다. **VZ-G-01~05 5건을 신설했습니다.**

이걸로 8/25부터 비어 있던 「명령 의도 해석의 주인」 구멍도 같이 메워졌습니다 — 발화 텍스트를 마일스톤으로 쪼개는 VZ-G-01이 곧 의도 해석이기 때문입니다.

대신 이 문서 첫 줄의 전제가 바뀝니다. “가시화는 데이터를 직접 만들지 않습니다”는 관측값에만 해당하고, **임무 계층은 이제 우리가 만듭니다.**

2026-08-27 개정 (1차) — 요구사항 엑셀 신판을 반영했습니다.

- **AI가 계획을 만들고 검증하던 요구사항이 통째로 사라졌습니다** (AI-D-01·02·04 삭제). → 2차 개정에서 가시화가 인수하는 것으로 정리됐습니다.
- **학습 계열도 사라졌습니다** (AI-L-01~03). VZ-U-06은 “그룹 확인 → 학습”이 아니라 “후보 개별 확인 → 분류 갱신”으로 좁아집니다.
- 하드웨어 주기가 여럿 바뀌었습니다 — 액추에이터 진행 200 ms → **50 ms**, 장치 현황 5초 → **10초**, 관측 export 15초 → **60초**, 오프라인 판정 즉시 → **약 4초 후**.
- 8/26 회의로 **디버깅 계열 VZ-D-01~08을 신설했습니다.**

2026-08-25 개정 — 상대 파트 시트 변경과 구현 진척을 반영했습니다.

- AI-C-14 **미디어 평면**(영상이 세 번째 평면이 됨) · AI-C-08 미디어 어댑터
- HW-R-04 **온디바이스/엣지 인지 급 분리** · AI-E-04·AI-S-02 선택 기능화
- AI-D-03이 「명령 의도 해석」에서 다른 요구사항으로 바뀌면서 **의도 해석의 주인이 사라진 것**(VZ-L-01~03)
- BE-T-06 **캐시 범위 회신**(채널 단위) · BE-T-03 브라우저 결속 해소
- Unity 트윈 뷰어 구현(VZ-U-02)

개정 전 PDF는 [_archive/](#)로 옮겼습니다.

엑셀 시트의 VZ-* 46개를 **말로 설명할 수 있는 형태로** 푼 문서.

46개가 전부 새로 만들 것은 아닙니다. 기존 프로토타입을 폐기하지 않고 **한 앱으로 합칩니다.** 기존 탭을 새 앱으로 이식하고, 지금 만드는 「임무 설계 및 디버깅」이 그중 탭①이 됩니다. 그래서 요구사항 46개가 **하나도 죽지 않습니다.**

- ① 임무 설계 및 디버깅 ← 지금 만드는 것
② 구역 현황판 ③ 제어 패널 ④ 지표 조회 ⑤ 영상 오버레이 ⑥ 파이프라인 편집기(동결)

배치

탭① 임무 설계 및 디버깅

공유 계층 (수신·명령·권한·표기)

탭 ②~⑤

탭 ⑥ 파이프라인(동결·시연용)

Unity 트윈 뷰어 (별도 앱)

탭①이 대체 · 보류

항목마다 엑셀의 **통합 앱 배치** 열에 어디로 가는지 표시해 뒀습니다. 전체 구조는 요
구사항정의서 §3.1에 있습니다. 각 항목은 ① 한 줄로 말하면 ② 쉽게 풀면 ③ 왜 이
주기인가 ④ 안 하면 어떻게 되나 순서.

0. 전체를 한 문장으로

**“가시화는 관측값을 만들지 않습니다. 받아서 그립니다.” “다만 임무 계층은 우리
가 만듭니다 — 발화를 마일스톤·태스크로 쪼개고, 검증하고, 평가합니다.”**

(2026-08-27 2차 개정 — 아래 원문은 관측값에 대해서는 그대로 유효합니다. 임무 생성은
VZ-G 계열을 보세요.)

여기서 두 가지가 따라 나옵니다.

첫째, 가시화 뷰어는 아무 데도 직접 붙지 않습니다. 문이 딱 네 개뿐이에요 — 실시간은 **WS
게이트웨이**, 지표는 **질의 프록시**, 감사는 **조회 API**, 구성은 **레지스트리 조회**. 브로커나 DB에
직접 접속하지 않습니다. 이유는 단순해요. 뷰어는 사용자 손 안에서 돌아가니 열쇠를 쥐여주
면 안 됩니다. **브라우저라서가 아닙니다** — 나중에 트윈 화면을 네이티브 앱으로 만들어도 똑
같이 백엔드를 거칩니다. 권한·감사·집약 경계를 지킬 수 있는 곳이 백엔드뿐이라서요.

둘째, 판단은 다른 파트가 하고 저는 표시합니다. 좌표 변환은 백엔드, 명령 확정 판정도 백엔
드, 음성 의도 해석과 계획 생성은 AI. 제가 하는 건 **받은 걸 사람이 알아볼 수 있게 만드는 것**
과 **사람이 한 행동을 정확히 넘기는 것**입니다.

주기를 정할 때 쓴 원칙도 하나입니다 — **“원천이 주는 것보다 자주 그려봐야 새 그림이 없
다.”** 그래서 대부분의 숫자는 하드웨어·AI가 정한 주기에서 따왔습니다.

구성 — 수신 11 · 송신 5 · 화면 7 · LLM/음성 4 · 공통 6 = **33개**

지금 33개 중 22개가 실제로 돕니다. 데이터가 들어와 화면에 당기까지의 뼈대 전부와
Unity 트윈 뷰어까지입니다. 남은 11개는 그 뼈대 위에 얹는 화면들과, 상대 파트의 답을 기
다리는 것들이에요.

그중 **“자리만 열어둔 것”은 이제 하나**(VZ-I-11 구독 범위 한정)뿐입니다. 원래 넷이었는데 집
약 표기(VZ-C-03)와 권한 범위(VZ-C-04)는 백엔드 계약이 확정되면서 **실사용으로 승격**됐고,

설계 규모(VZ-C-05)와 원천 종류(VZ-C-06)는 자리 확보가 아니라 **팀 합의를 기다리는 항목**입니다.

수신 (VZ-I) — 받아오는 것 11개

VZ-I-01 · 실시간 상태 구독

한 줄로 말하면 > “로봇·센서 상태를 백엔드를 거쳐 실시간으로 구독합니다. 상태는 한 덩어리가 아니라 세 층으로 받습니다.”

쉽게 풀면 뷰어는 장치가 쓰는 전송 프로토콜에 직접 붙지 않습니다. 백엔드가 중간에서 받아 웹소켓으로 넘겨줘요. (브라우저는 아예 못 붙고, 네이티브 앱은 붙을 수야 있지만 **붙으면 안 됩니다** — 권한과 감사가 백엔드를 우회하니까요.) 구독할 때 **채널 주소가 아니라 “어느 개체, 어느 노드, 어떤 채널”**로 요청합니다. 전송 규격이 바뀌어도 제 화면이 안 깨지게요.

왜 이 주기인가 로봇은 임무 중 50ms(초당 20번)로 상태를 올립니다. 그런데 **올 때마다 화면을 다시 그리면 오히려 끊깁니다**. 그래서 데이터는 다 받되 **화면 반영만 0.1초 단위로 묶어서** 합니다.

다만 원천마다 속도가 다릅니다. 로봇은 50ms로 촘촘한데, **센서는 평소 1분에 한 번이고 임계값을 넘거나 값이 급변할 때만 즉시** 올라옵니다. 그래서 평소엔 한산하다가 이벤트 때 몰리는 패턴이에요.

안 하면 초당 20번 렌더링하다가 화면이 버벅입니다. 반대로 데이터를 버리면 이동 궤적이 끊겨 보이고요.

VZ-I-02 · 재접속 시 현재값 확보

2026-08-27 개정 — 대기 중 로봇 보고가 5초에서 **1초**로 바뀌었습니다(HW-R-03). 빈 화면 위험은 줄었지만 평소 센서 1분(HW-S-02)은 그대로라 **최악 1분**은 변함이 없습니다.

한 줄로 말하면 > “화면을 새로 열면 그 순간 현재 상태를 한 번 받아야 합니다. 안 그러면 빈 화면입니다.”

쉽게 풀면 발행-구독 방식은 “지금부터 오는 것”만 줍니다. 과거 값은 안 줘요. 그러니 화면을 새로 열면 **다음 데이터가 올 때까지 아무것도 못 그립니다**.

얼마나 비어 있나 — 이게 설득 포인트입니다. - 대기 중인 로봇은 5초마다 보고 → 최대 5초 공백 - 평소 센서는 1분마다 보고 → **최대 1분 빈 화면**

해결됐습니다 백엔드가 마지막 상태를 캐시하고 있다가 **구독하는 순간 한 번 던져주는 방식**으로 확정됐습니다. 뷰어가 어차피 백엔드를 거치니 경로가 하나로 끝나는 쪽이 낫다는 판단이에요.

범위도 답을 보냈습니다 — 다만 “전부 캐시”는 안 됩니다 백엔드가 “어디까지 캐시할까”를 물어와서(BE-T-06) 답을 정리했는데, 결론은 필드가 아니라 채널마다 판단이 갈린다였습니다.

판단	무엇	왜
줘야 하는 것 (7)	상태 · 계측값 · 액추에이터 · 제어 잠금 · 계획 · 계획 진행 · 영상 규격	없으면 화면이 “모른다”를 그리는데 서버는 알고 있습니다
주면 안 되는 것 (3)	지난 명령 결과 · 영상 프레임 · 탐지 결과	재접속하면 화면이 거짓말을 합니다
안 줘도 되는 것 (2)	하트비트 · 지표	이미 상태에 실려 오거나 별도 조회 경로가 있습니다

가운데가 요점입니다. 지난 명령 결과가 **방금 온 것처럼** 뜨고, 옛 프레임 한 장이 **정지 화면인 데 현재처럼** 보이고, 탐지 박스가 지금 화면에 없는 프레임을 가리켜 **영동한 자리에** 뜹니다. 관제에서 이건 빈 화면보다 위험합니다.

하나 더 요청했습니다 — **캐시된 값의 시각은 원래 발행 시각 그대로**. 내려주는 순간으로 다시 찍으면 1분 전 값이 방금 값처럼 보이고, 직후 서버가 “오래됨” 판정을 내리면 “**방금 왔는데 판단 불가**” 라는 앞뒤 안 맞는 상태가 됩니다.

VZ-I-03 · 구성(레지스트리) 조회

한 줄로 말하면 > ““지금 뭐가 있어야 하는지” 명단을 받아야 합니다. 반 명부 같은 거예요.”

쉽게 풀면 출석부가 없으면 **결석한 학생을 표시할 수 없습니다**. 온 학생만 보이니까요. 마찬가지로 배포되지 않은 로봇은 데이터를 안 보내니 상태 메시지만으로는 **화면에 영원히 안 나타납니다**.

받아야 하는 것 — 있어야 할 개체 목록, 각 개체가 어느 구역 소속인지, 구역 원점이 전역 어디인지, 그리고 표시 이름·별칭.

왜 이 주기인가 명단은 장치를 등록하거나 배포하거나 구역을 옮길 때만 바뀝니다. **처음에 한번 + 바뀌면 알려주기**면 충분합니다.

안 하면 “의도적으로 안 켜 것”과 “고장 나서 안 오는 것”을 구분할 수 없습니다. **구현 자체가 불가능**해집니다.

VZ-I-04 · 지표 질의

2026-08-27 개정 — 주기가 어긋났습니다. 하드웨어 지표 export가 15초에서 **60초**로 바뀌었는데(HW-C-05), 백엔드 요약 pull은 여전히 15초입니다(BE-S-03). 그래서 하드웨어 계열 지표는 **15초로 질의해도 같은 점이 네 번 반복**됩니다. 화면이 “갱신되지 않는다”로 오해되지 않게 지표별 원천 주기를 함께 표기하되, 어느 쪽 주기에 맞추지는 팀 합의가 필요합니다.

한 줄로 말하면 > “그래프용 과거 데이터는 백엔드 프록시를 통해 조회합니다.”

받는 게 원본이 아닙니다 평시에는 구역 단위로 요약된 값이 올라옵니다. 원본은 옛지에 남아 있어요. 원본이 필요한 구간만 프록시가 옛지까지 가서 대신 가져옵니다.

왜 이 주기인가 백엔드가 그 요약을 15초마다 당겨옵니다. 그러면 제가 1초마다 물어봐도 대부분 아까 그 값이에요. 그래서 기본 15초로 맞췄습니다.

단, 센서가 이벤트 모드로 들어가면 보고 주기가 1초로 올라갑니다. 홍수 대응 골든타임이니까요. 그때는 저도 1초로 따라갑니다.

조회 범위가 길면(1시간 이상) 점 간격이 넓어져서 30초로 늦춥니다.

VZ-I-05 · 감사 이력 조회

한 줄로 말하면 > “‘이 장비를 마지막으로 누가 조작했나’와 그 판단 근거를 화면에 띄우려면 감사 기록을 되읽어야 합니다.”

쉽게 풀면 제가 감사를 쓰지는 않지만 읽기는 해야 합니다. 쓰기만 있고 읽기가 없으면 책임 추적이 화면에서 확인되지 않아요.

왜 이 주기인가 감사는 이미 지나간 기록입니다. 패널을 열 때만 조회합니다. 진행 중인 명령의 상태 변화는 결과 푸시로 따로 오니까 중복이고요.

VZ-I-06 · 영상 스트림 표시

한 줄로 말하면 > “카메라 영상은 뷰어가 재생할 수 있는 형태로 바뀌서 받아야 합니다.”

쉽게 풀면 — 영상이 세 갈래입니다

무엇

로봇 관제용 영상

고정 비전센서 영상

로봇 인식용 프레임

세 번째는 **AI가 먹는 것이라 제 화면과 무관합니다**. 원래 하나로 뭉쳐 있던 걸 하드웨어가 분리해줬어요.

영상 픽셀은 업무·관측 경로와 분리된 세 번째 미디어 평면으로 갑니다(AI-C-14). 메시지 브로커나 OpenTelemetry에 영상을 직접 신지 않습니다. AI-C-08의 GStreamer 어댑터가 AI 입력까지는 담당하지만, 브라우저·Unity 뷰어로 내보내는 WebRTC/HLS 등의 출력 분기와 소유 파트는 아직 정해지지 않았습다.

왜 이 주기인가 원본이 15fps니 그 이상 요구해봐야 받을 게 없습니다. 그리고 카메라 전부를 상시로 틀면 무선 대역폭과 뷰어 디코딩을 동시에 낭비하니, **열어놓은 패널만** 받습니다.

VZ-I-07 · 탐지 결과 오버레이 정합

한 줄로 말하면 > “AI가 찾은 물체 박스를 영상 위에 겹치려면, 그 박스가 **몇 번째 사진** 것인지 알아야 합니다.”

쉽게 풀면 사진에 번호를 붙여 보내고, AI가 결과를 줄 때 **그 번호를 그대로 돌려주는** 겁니다. 그래야 “4821번 사진의 결과”를 4821번 사진 위에 그릴 수 있어요.

안 하면 엡지 정밀 인지가 0.2초 걸리고 영상이 15fps니까, 결과가 돌아왔을 때 화면은 이미 **3프레임 앞서 있습니다**. 번호 없이 도착 순서대로 그리면 박스가 항상 **사람이 지나간 자리**에 남아요.

말이 아니라 숫자로 재봤습니다 (목 게이트웨이 실측)

설정

번호 맞춰 그림

번호 무시 · 지연 0.2초

번호 무시 · 지연 0.5초

온디바이스 값이 지연과 무관하게 30px 안팎인 이유는 온보드 판단이 **0.06초로 고정**돼 있기 때문입니다 — 안전 판단은 엡지 왕복을 기다리지 않습니다.

여기에 하나 더 — bbox 좌표가 픽셀인지 0~1 비율인지, 원점이 어디인지, 기준 해상도가 뭔지도 정해져야 합니다.

해결됐습니다 프레임 참조 형식이 **공통 규약으로 확정**됐습니다(원천 식별자 · 촬영 시각 · 시퀀스 번호). AI와 하드웨어가 같은 형식을 쓰기로 정리돼서, 이 기능의 최대 리스크가 사라졌습니다.

결과의 급이 같았습니다 (2026-08-24 · HW-R-04 재작성)

로봇 온보드는 **라즈베리파이와 카메라뿐**입니다. 거리를 재는 센서가 없어요. 그래서 온디바이스는 “**진행영역에 뭔가 있고 가까워지고 있다**” 까지만 말하고, **무엇인지 분류하지 않습니다**. 분류·추적은 엡지 AI에서 옵니다.

무엇을 내나

추적 번호

분류 신뢰도

필수/선택

둘을 같은 모양으로 그리면 안 됩니다. 관제사가 거친 판단을 정밀 판단으로 읽고, “**0.9면 확실하다**” 를 두 결과에 똑같이 적용하게 됩니다. 온디바이스에는 애초에 그 숫자가 없고요. 그래서 화면이 색·선·라벨을 갈라 그림니다.

오티지 결과가 안 오면 화면이 그 사실을 말합니다. 다만 알 수 있는 건 “안 온다”까지고 **미배포 인지 장애인**지는 구분 못 합니다 — 그걸 알려주는 경로가 계약에 없어요(아래 「지금 걸려 있는 것」 참고).

참조 프레임이 이미 버퍼에서 빠졌다면, 현재 프레임과 비교한 값을 정합 성공처럼 표시하지 않고 **참조 프레임 없음**으로 알립니다.

VZ-I-08 · 위험도 상태 표시 (신설)

한 줄로 말하면 > “지금 이 평시인지 관찰인지 경보인지, 그리고 왜 그렇게 판단했는지를 화면에 보여줍니다.”

쉽게 풀면 AI가 강우·수위 같은 신호를 보고 **평시 / 관찰 / 경보 / 복구** 네 단계로 상황을 판정합니다. 여기에 위험도 점수와 **판단 근거, 권고 조치**가 함께 옵니다.

중요한 건 근거예요. “위험합니다”만 뜨면 관제사가 판단할 수 없습니다. **무엇 때문에 그렇게 봤는지**가 같이 와야 하고, 그게 제어의 근거가 됐다면 제어 이력에서도 되짚을 수 있어야 합니다.

왜 이 주기인가 평시엔 위험도가 거의 안 변해서 계속 물어볼 이유가 없습니다. 대신 **상태가 올라가는 순간은 화면 전환·경보를 유발**하므로 즉시 도달해야 합니다.

VZ-I-09 · 객체 추적 및 궤적 표시 (신설)

한 줄로 말하면 > “물체를 한 장면씩 보는 게 아니라, 같은 물체로 이어 붙여 궤적까지 보여줍니다.”

쉽게 풀면 탐지 박스만 있으면 매 프레임 **처음 보는 물체**입니다. 추적 식별자가 붙어야 “아까 그 사람이 이쪽으로 왔다”가 되고, 속도·이동 방향·지나온 경로를 그릴 수 있어요.

여기에 두 가지가 더 붙습니다. - **카메라 간 연결** — 여러 카메라에 걸친 같은 물체를 하나로 봅니다. 이 기능은 선택 사항이므로 연계가 없으면 억지로 묶지 않고 camera-02:trk-01처럼 **소스별 추적을 유지**합니다. - **불확실성 표시** — 분류 신뢰도를 화면에서 구분해 보여줍니다. 확실한 것과 애매한 것이 똑같이 보이면 안 됩니다. 다만 **온디바이스 결과에는 분류 신뢰도가 없으므로**(분류를 안 함) 그 자리에 숫자를 지어내지 않고 「분류 없음」으로 둡니다.

단기 궤적 **예측**은 선택적 확장으로 뒀습니다.

왜 이 주기인가 추적은 탐지에서 파생되니 **원천보다 자주 받을 수 없습니다**. 따로 주기를 두면 궤적이 실제 위치와 어긋나고요.

VZ-I-10 · AI 실패 이벤트 알림 (신설)

한 줄로 말하면 > “AI가 실패했을 때는 지표가 아니라 알림으로 즉시 받습니다.”

쉽게 풀면 정상일 때의 성능 지표(VZ-I-04)와 **실패는 성격이 다릅니다**. 실패는 드물게 일어나지만 일어난 순간이 중요해요. 무엇이·어느 컴포넌트에서·어떤 모델 버전으로·무슨 입력에서 실패했는지가 함께 옵니다.

왜 별도 경로인가 지표 질의는 15초마다 물어봅니다. 실패를 여기 실으면 **최대 15초 늦게 알게 됩니다**. 그래서 도착 즉시 알리는 경로로 분리했습니다.

VZ-I-11 · 구독 범위 한정 (자리 확보)

“자리만 열어둔 것”이 뭔가 — 지금은 쓰지 않지만, 계약에 **필드 자리를 미리 비워 두는** 요구사항입니다. Zone을 선택 항목으로 남겨둔 것과 같은 논리예요. 지금 자리 하나 여는 비용은 **필드 하나**지만, 나중에 계층을 끼우는 비용은 **전 파트로 전파**됩니다.

한 줄로 말하면 > “지금은 전부 구독하지만, 대상이 많아지면 ‘보고 있는 것만’ 구독할 수 있게 자리를 열어둡니다.”

쉽게 풀면 지금은 구역 하나에 장치가 수십 개라 전부 구독해도 됩니다. 그런데 구역이 늘어나면 **뷰어가 받는 양 자체가 문제**가 돼요.

여기서 헛갈리기 쉬운 게 있습니다. 화면 갱신을 100ms로 묶는 건(VZ-I-01) **그리는 부하만** 줄입니다. **받는 양은 그대로**예요. 그러니 애초에 구독할 때 범위를 좁히는 수밖에 없습니다.

지금은 안 씁니다 시연 규모에서는 전체 구독이 더 단순합니다. 다만 나중에 구독 방식을 바꾸려면 백엔드 게이트웨이와 화면 전체가 같이 움직여야 해서, **요청 형식에 자리만** 남겨둡니다.

송신 (VZ-O) — 내보내는 것 5개

VZ-O-01 · 제어 명령 발행

한 줄로 말하면 > “사용자 조작을 ‘무엇을 하라’는 추상 명령으로 백엔드에 넘깁니다. 장비 명령으로 바꾸는 건 백엔드가 합니다.”

쉽게 풀면 저는 수문 개방까지만 보냅니다. 그게 실제 장비 명령이 되는 건 백엔드가 어휘집을 보고 번역해요. **장비가 바뀌어도 제 화면은 안 고쳐도 됩니다**.

주문번호는 백엔드가 붙입니다 전 파트가 함께 쓰는 상관 키(command_id)는 **백엔드가 발급**합니다. 제가 붙이는 게 아니예요. 그럼 발행하는 순간에는 뭘로 추적하나 — **제 요청 식별자**를 먼저 붙여 보냅니다.

버튼을 누른 직후 화면은 그 요청 식별자에 “진행중”을 걸어두고, 백엔드가 상관 키를 응답으로 내려주면 **그때부터 그 키로** 결과와 감사를 따라갑니다. 두 식별자의 짝은 백엔드가 들고 있어요.

같이 실어 보내는 것 하나 더 — **완료 시각**입니다. 유통기한이라 시간이 지난 명령은 실행하면 안 됩니다.

왜 이 주기인가 사용자가 누를 때 나가는 거라 주기가 없습니다. 모아서 보내면 긴급 제어가 늦어지고, 만료 시각의 의미도 사라지고요.

VZ-O-02 · 명령 결과 표시

2026-08-27 개정 — 진행 보고가 4배 촘촘해졌습니다. 액추에이터 진행 상태 보고가 200 ms(5 Hz)에서 **50 ms(20 Hz)**로 바뀌었습니다(HW-A-01·HW-A-04). 데이터는 전량 받되 **화면 반영은 VZ-I-01과 같은 100 ms 창으로 병합합니다** — 진행 막대를 20 Hz로 다시 그릴 이유가 없고, 상태와 명령 진행이 서로 다른 창으로 갱신되면 같은 화면에서 어긋나 보이기 때문입니다.

한 줄로 말하면 > “버튼을 눌렀다고 바로 ‘열림’으로 바꾸지 않습니다. 실제로 열렸다는 확인이 와야 바꿉니다.”

쉽게 풀면 — 네 단계로 옵니다

1. **수신 확인** — “명령 받았다”
2. **수행 중** — 물리적으로 움직이는 동안 **200ms마다 진행 상태**가 옵니다
3. **물리 상태 변화** — 실제로 상태가 바뀌었다
4. **완료 / 실패**

화면에는 **진행중 · 확정 · 실패** 세 가지로 보여주되, 2번 덕분에 **움직이는 동안 멈춘 것처럼 보이지 않게** 진행 상태를 함께 표시합니다. 수문처럼 되돌리기 어려운 건 1번이 아니라 3~4번까지 기다려야 합니다.

안 하면 버튼을 누르는 순간 열린 걸로 표시했는데 실제로는 실패 — 화면과 현실이 어긋납니다. 관제 화면에서 제일 위험한 상황입니다.

VZ-O-03 · 책임소재(감사) 필드 전달

한 줄로 말하면 > “누가·어떻게 시켰는지를 명령에 같이 실어 보냅니다. 기록을 쓰는 건 백엔드입니다.”

쉽게 풀면 — 왜 제가 직접 안 쓰나 세 가지 이유가 있습니다.

1. **위조** — 뷰어가 직접 쓰면 “누가 했다”를 클라이언트가 자기 입으로 말하는 셈입니다. 남의 이름을 넣어도 서버는 받아 적어요.
2. **시계** — 장치들은 NTP로 시각을 맞춥니다. 그런데 **뷰어를 돌리는 단말에는 그런 보장이 없어요.** 사용자 PC 시계로 기록하면 법적 추적성을 주장하기 어렵습니다.
3. **유실** — 명령은 실행됐는데 탭을 닫아서 기록이 안 남을 수 있습니다. **감사에서는 최악의 실패**입니다.

그래서 명령과 **한 요청에 같이** 실어 보내고, 조작자와 시각은 백엔드가 토큰과 서버 시계에서 채웁니다.

지금 걸린 문제 음성으로 말했고 LLM이 해석한 경우, 현재 어휘로는 둘 중 **하나만 골라야 합니다**. 어느 쪽을 버려도 사고 원인(잘못 들었나 / 잘못 해석했나)을 구분할 수 없어서, **입력 수단과 판단 주체를 따로 기록하자고 요청 중**입니다.

VZ-O-04 · 자체 관측 지표 발행

한 줄로 말하면 > “제 화면이 얼마나 느린지는 제가 직접 관측 시스템에 보냅니다.”

쉽게 풀면 다른 데이터는 백엔드를 거치지만 **자기 성능 지표만은 직접** 보냅니다. 남을 관찰한 게 아니라 자기 자신에 대한 값이니깐요. AI도 똑같이 합니다.

왜 60초인가 추세만 보면 되는 값이라 촘촘할 이유가 없습니다. 1초마다 보내면 저장 비용만 늘고 판단은 안 달라져요.

VZ-O-05 · 제어 잠금 상태 표시

2026-08-27 개정 — 통신 장애 판정이 3회 미수신(약 3초)에서 **4회 미수신(약 4초)**로 바뀌었습니다(HW-A-05). 잠금이 뜨기까지 1초 더 걸립니다.

한 줄로 말하면 > “통신이 끊긴 장비는 제어 버튼을 잠그고, 잠겼다는 걸 화면에 보여줍니다.”

쉽게 풀면 장비와 연결이 끊기면 **원격 제어가 막힙니다**. 안전 상태를 유지하고, 통신이 돌아와도 **실제 상태를 다시 확인하기 전까지는** 제어를 열지 않아요.

문제는 그 사실을 화면이 모르면 **관제사가 눌러도 실행되지 않는 버튼을 계속 누른다는** 겁니다. 그래서 잠금 상태와 사유를 표시합니다.

왜 이 주기인가 잠금은 통신 두절 판정(하트비트 3회 연속 미수신, 약 3초)과 복구 시점에만 바뀝니다. 대신 **늦게 알면 곤란해서** 전환되는 즉시 받습니다.

화면 (VZ-U) — 그리는 것 7개

VZ-U-01 · 상태 4종 구분 표시

2026-08-27 개정 — 주기 둘이 바뀌었습니다. 구역별 장치 현황판은 5초에서 **10초**로 낮췄습니다 (HW-C-04가 그렇게 바뀌었고, 그보다 자주 그려도 같은 값입니다). 그리고 오프라인 감지가 “즉시”에서 **1 Hz 하트비트 4회 미수신(약 4초)** 후로 바뀌었습니다(HW-S-07). 화면은 판정이 오는 즉시 반영하지만 **끊김과 표시 사이에 약 4초가 구조적으로 존재합니다**. 이 지연을 감추면 “방금까지 정상이었다”는 오해를 만들기 때문에 화면에 드러냅니다.

한 줄로 말하면 > “장비 상태를 네 가지로 구분합니다 — 정상 / 장애 / 일부러 안 켜 것 / 판단 불가.”

쉽게 풀면 — 왜 세 층으로 받아 상태를 아는 주체가 셋입니다.

누가 아는가

기기 자신

서버

배포 시스템

이걸 하나로 합치면 “**연결은 멀쩡한데 기기가 고장난 상태**”를 표현할 수 없습니다. 연결됐으니 정상으로 보이거든요.

판단 불가가 뭔가 전원은 들어와 있는데 **시계 바늘이 안 움직이는** 상태입니다. 연결은 살아 있는데 값이 갱신되지 않는 거예요.

액추에이터는 어휘가 다릅니다 수문·펌프는 대기 / 동작 중 / 완료 / 오류 / 확인 불가처럼 **자기만의 상태**를 갖습니다. 이건 표준 3층과 별개인 도메인 어휘로 다릅니다. 공통 컴포넌트가 하드코딩하면 안 돼요.

VZ-U-02 · 디지털 트윈 위치 렌더링

한 줄로 말하면 > “로봇 위치를 3D 화면에 놓습니다. 좌표 변환은 백엔드가 하고 저는 배치만 합니다.”

쉽게 풀면 로봇은 **자기 구역 기준**으로 좌표를 말합니다. “여기서 2m 앞”처럼요. 그걸 전체 지도 좌표로 바꾸는 건 백엔드가 합니다. 저는 변환 코드를 **아예 안 만듭니다**.

왜 보간을 하나 50ms마다 오는 좌표를 그대로 찍으면 로봇이 **초당 20번 순간이동**하는 것처럼 보입니다. 서버에 더 자주 보내달라고 하면 무선이 못 버티고요. 그래서 **받은 점 사이를 제가 채워서** 부드럽게 만듭니다.

해결했습니다 좌표계 규약과 전역 변환이 **백엔드 단독 책임으로 확정**했습니다(y=위·바닥 x-z·미터). 하드웨어 시트에서 빠져 있던 게 백엔드 쪽에 자리를 잡았어요. 출처 구분(자기추정 / 앵커 보정 / AI 인지)도 그대로 남아 있습니다.

구현했습니다 Unity 6 별도 클라이언트가 웹과 같은 게이트웨이·레지스트리 계약을 읽어 로봇을 공간에 배치하고, 50ms 좌표 사이를 보간합니다. 상태 색상, 집약 표기, 원천 종류와 handedness 진단도 웹과 같은 의미로 확인합니다. 아직 Unity **시뮬레이터** 단계는 아니며 현재는 뷰어입니다.

VZ-U-03 · 추상화 계층 뷰

한 줄로 말하면 > “같은 대상을 결심자·운영자·개발자 시점으로 바꿔 봅니다.”

쉽게 풀면 지휘관은 빨강/초록만 보면 되고, 운영자는 어느 장비가 문제인지, 개발자는 원본 수치를 봐야 합니다. **같은 데이터를 세 깊이로** 보여주는 겁니다.

헷갈리기 쉬운 점 구역을 좁혀 들어가는 것(전체 → 구역 → 로봇)과는 **다른 축**입니다. 같은 로봇을 보면서도 판단 수준을 바꿀 수 있어야 해요.

주의할 것 AI가 준 위험도 같은 값은 이미 요약된 값입니다. 그걸 원본인 줄 알고 또 평균 내면 틀린 숫자가 나와요.

VZ-U-04 · 노드 편집기 구성 반영

한 줄로 말하면 > “화면 구성을 블록 연결로 만들고, ‘반영’ 버튼을 눌러야 실제 화면에 적용됩니다.”

쉽게 풀면 데이터를 어떻게 해석해 보여줄지를 코드로 미리 고정하지 않고 쓰는 시점에 조합합니다. 편집 중에 자동 반영되면 미완성 구성이 운영 화면에 나가버리니, 명시적으로 눌러야 적용됩니다.

반영은 제어 명령급으로 봅니다 화면 구성을 바꾸는 것도 “시스템 판단에 영향을 주는 행위”라 감사 대상입니다. 다만 편집기가 아직 없으니 자리만 만들어두고 실제 기록은 나중에 시작합니다.

VZ-U-05 · 서브태스크 진행 상태 표시

2026-08-27 개정 — 상대가 사라졌습니다. 이 구간 정보를 만들어 주던 AI-D-01(서브태스크 생성)과 AI-D-02(검증)가 엑셀 신판에서 삭제됐습니다. 화면은 그대로 필요한데 채울 데이터를 만드는 파트가 없습니다. 담당이 정해지기 전까지는 목 데이터로만 성립합니다.

한 줄로 말하면 > “승인된 계획이 어디까지 갔고 어디서 실패했는지를 단계별로 보여줍니다.”

쉽게 풀면 계획은 여러 구역에 걸쳐 구간으로 쪼개져 하달됩니다. 실패했을 때 어느 구간에서 왜인지는 화면에서만 알 수 있어요. 구간을 블록으로 늘어놓고 완료·실패·대기를 표시합니다.

왜 이 주기인가 상태가 바뀌는 건 하달·시작·완료·실패 네 시점뿐입니다. 수행 결과가 로봇 상태 데이터에 실려 오므로 별도 폴링이 필요 없어요.

VZ-U-06 · 미확인 객체 후보 확인 (2026-08-27 축소)

개정 — 목적이 좁아졌습니다. 유사한 것끼리 묶어 주던 AI-L-02(군집화·대표선정)와 학습 반영 경로(AI-L-01·AI-L-03)가 전부 삭제됐습니다. 미확인 객체를 후보로 남겨 두는 AI-S-04는 살아 있어서, 확인 절차 자체는 성립합니다.

한 줄로 말하면 > “AI가 ‘이게 뭔지 모르겠다’고 남겨 둔 것을 사람이 이름 붙여 돌려줍니다.”

쉽게 풀면 AI가 처음 보는 물체를 만나면 분류를 확정하지 못하고 미확인 상태로 남겨 둡니다. 화면에서 “이건 소화전이야”라고 알려주면 그 분류 상태가 갱신됩니다.

무엇이 달라졌나

전달 단위
확인 결과가 가는 곳

그룹으로 묶어 주지 않으면 확인 요청이 건마다 오게 되어 사용자 부담이 커집니다. **묶는 일을 누가 할지가 새로 비었습니다** — 가시화가 화면에서 묶어 보여주는 방법도 있지만, 그건 유사도 계산을 화면이 떠안는 것이라 간단하지 않습니다.

중요한 원칙 사람 확인 없이는 분류가 확정되지 않습니다. 이 원칙은 그대로입니다.

VZ-U-07 · 계획 승인 및 근거 표시

2026-08-27 개정 — 승인할 대상을 만드는 파트가 없습니다. 계획 생성(AI-D-01)·검증(AI-D-02)·재생성 판단(AI-D-04)이 전부 삭제됐습니다. 승인 절차와 화면은 그대로 필요하지만, **무엇을 승인할 것인지가 만들어지지 않습니다.** 아래 설명은 생산 주체가 정해진다는 전제에서 유효합니다.

한 줄로 말하면 > “AI가 만든 계획은 사람이 승인해야 실행됩니다. 그 판단 근거를 펼쳐서 보여줍니다.”

쉽게 풀면 계획은 여러 단계를 거칩니다 — **전역 임무 → 구역별로 쪼개기 → 구간별 계획 생성 → 규칙 검증.** 검증을 통과해도 **바로 실행되지 않습니다.** 사람이 승인해야 해요.

계획을 만들고 검증하는 건 AI지만, **저에게 가져다주고 승인 결과를 받아 가는 건 백엔드입니다.** 승인된 것만 로봇으로 나갑니다.

그러려면 화면이 **왜 이 계획인지**를 보여줘야 합니다. 어떤 임무에서 나왔고, 어느 구역을 어떤 순서로 지나고, 무슨 검증을 통과했고, 어떤 버전의 계획 생성기가 만들었는지까지요.

왜 필수인가 승인 없이 자동 실행하면 **책임소재가 성립하지 않습니다.** 사고가 났을 때 “AI가 했다”로 끝나버려요.

주의 이건 음성 명령과 무관합니다. **모든 계획이** 이 절차를 거칩니다. 음성은 그 입구 중 하나일 뿐이에요.

LLM · 음성 (VZ-L) — 4개

VZ-L-01 · 음성 입력(STT) 및 텍스트 전달

한 줄로 말하면 > “말을 글자로 바꾸는 것까지가 제 몫이고, 그 뜻을 푸는 건 AI가 합니다.”

경계가 확정됐습니다 음성을 글자로 바꾸는 것까지가 제 몫이고, **AI는 텍스트를 받습니다.** 한 동안 모호했는데 정리됐어요.

그런데 받은 텍스트의 뜻을 풀 사람이 없어졌습니다 △ (2026-08-24)

원래 AI 시트에 「관리자 명령 의도 해석」 (AI-D-03)이 있었습니다. 제 요구사항 다섯 곳이 그걸 가리키고 있었어요. 그런데 그 번호가 「**의사결정 보조 정보 요청**」이라는 전혀 다른 요구사항으로 바뀌었습니다. 번호는 그대로라 자동 검사에는 안 걸립니다 — 뜻만 조용히 바뀐 거예요.

지금 가장 가까운 건 「생성형 AI 보조 실행」 (AI-C-09)입니다. 필수 입력에 **작업 종류 · 입력 텍스트 · 기대 출력 형식**이 있어서 여기에 태우면 구조적으로는 맞습니다. **다만 이건 통로이지 기능이 아닙니다** — “생성형 AI를 어떻게 부르는가”까지고, ① 그렇게 부르는 주체가 어느 행에도 없고 ② 결과가 어떤 형식으로 오는지 아무 데도 없습니다.

그래서 이 항목은 **[확인 요망]** 상태로 두었습니다. 소재가 정해지기 전에 화면을 만들면 며칠 뒤 다시 만들게 됩니다.

쉽게 풀면 음성은 새로운 명령 체계가 아니라 입력 수단입니다. 마우스 클릭 대신 말로 하는 것 뿐이에요. 그래서 기존 명령 경로를 그대로 탑니다. **백엔드 입장에서는 새 채널이 안 생깁니다.**

“로봇 1번”을 어떻게 알아듣나 사람은 go1-01이라고 말하지 않습니다. 그래서 레지스트리에서 **표시 이름과 별칭**을 같이 받아옵니다. 별도 시스템이 아니라 명부에 칸 하나 추가하는 정도예요.

왜 발화 끝날 때 한 번인가 계속 켜두고 들으면 관제실 잡음까지 전부 처리합니다. 오인식도 높고 연산도 낭비고요.

VZ-L-02 · 해석 결과 표시 및 수락

2026-08-27 개정 — 구멍이 하나에서 둘이 됐습니다. 원래도 “명령 의도를 해석하는 파트가 없다”가 걸려 있었는데, 이번에 **계획을 생성하는 파트까지 사라졌습니다** (AI-D-01 삭제). 해석과 계획 생성 둘 다 담당이 비어 있습니다.

한 줄로 말하면 > “AI가 해석하고 만든 계획을 ‘이렇게 하면 될까요?’로 보여주고, 사람이 눌러야 실행됩니다.”

쉽게 풀면 — 왜 해석을 AI에 넘겼나 해석기를 저와 AI가 각각 두면 **같은 말이 두 결과를 냅니다.** 그러면 문제가 생겼을 때 누구 탓인지 알 수 없어요. 해석과 계획 생성은 AI 한 곳으로 모으고, 저는 **전달하고 보여주고 수락받는 역할만** 합니다.

계획 생성은 주인이 있고, 의도 해석은 없습니다 △ 계획을 만들고 검증하는 쪽(AI-D-01·AI-D-02)은 시트에 그대로 있습니다. 문제는 그 앞 단계예요 — **말을 임무로 바꾸는 부분의** 담당이 비었습니다(VZ-L-01 참고).

이 화면이 그리는 것은 “*AI가 이렇게 해석했습니다*” 인데, **그 해석 결과가 어떤 모양으로 오는지가 정해지지 않으면 그럴 것이 없습니다.** 칸을 몇 개 두고 무엇을 채울지가 출력 형식에 달려 있어서, 지금 만들면 형식이 정해질 때 다시 만들게 됩니다. **[확인 요망]**

VZ-L-03 · 실행 전 2단 확인

2026-08-27 개정 — 2단 확인의 ②단계(해석이 맞는지)에서 보여줄 검증 결과를 만드는 **AI-D-02가 삭제**되었습니다. 당분간 ②단계는 해석 결과만을 대상으로 하고, 검증 결과 표시는 생산 주체가 정해진 뒤에 붙입니다. **2단 확인 자체는 그대로 필수**입니다.

한 줄로 말하면 > “두 번 확인합니다 — 내가 말한 게 맞는지, 그리고 그걸 이렇게 해석한 게 맞는지.”

쉽게 풀면 - 1차 — “제가 이렇게 들었는데 맞나요?” (잘못 들었을 수 있음) - **2차** — “그 말을 이 임무·경로로 해석했는데 맞나요?” (잘못 해석했을 수 있음)

오류가 날 수 있는 지점이 두 군데니 확인도 두 번입니다.

되돌릴 수 있는 건 봐줍니다 화면 전환이나 조회는 잘못돼도 다시 누르면 그만이라 1차만 합니다. 하지만 **제어는 예외 없이** 두 번 거칩니다.

신뢰도가 낮으면 아예 AI에 보내지도 않고 다시 말해달라고 합니다. “높으면 건너뛰자”는 안 이 있을 수 있는데, **사고는 애매한 구간에서 납니다.**

해석 기능이 아예 없는 배치라면 △ AI-C-09는 명시적으로 **선택 기능**입니다. 의도 해석을 거 기에 태우면, 배치에 따라 **음성 명령이 통째로 없는 시스템**이 됩니다. 그 자체는 나쁘지 않아요 — 제 VZ-C-02(“연결 못 하면 관련 기능만 비활성화”)와 방향이 같습니다.

문제는 **없을 때 화면이 무엇을 해야 하는지**입니다. 버튼을 숨길지, “지금은 음성을 쓸 수 없습니다”를 띄울지. 알려면 “**이 배치에 그 기능이 있다/없다**” 가 화면까지 와야 하는데 **그 경로가 없습니다.** 그래서 지금은 “해석 기능이 없는 배치에서는 음성 제어 경로를 비활성화한다”까지만 적어 두었습니다.

VZ-L-04 · 화면 구성 보조 LLM

한 줄로 말하면 > “제 쪽 LLM은 화면 만드는 걸 돕습니다. 로봇을 조종하는 해석은 안 합니다.”

쉽게 풀면 “온도 높은 구역 보여줘” 같은 걸 화면 구성으로 바꾸거나, 데이터를 요약해주는 용도입니다. **로봇 제어 의도 해석은 AI 담당**이고 여기 포함되지 않습니다.

중요한 전제 LLM이 안 붙어 있어도 **나머지 기능은 전부 동작해야 합니다.** LLM은 있으면 좋은 것이지 없으면 안 되는 게 아닙니다.

공통 (VZ-C) — 6개

VZ-C-01 · 권한(RBAC)과 역할 확인

한 줄로 말하면 > “화면에서 버튼을 가리는 건 편의고, 진짜 차단은 백엔드가 합니다.”

쉽게 풀면 뷰어 코드는 사용자가 고칠 수 있습니다. 그러니 **화면에서 숨기는 건 방어가 아니에요**. 실제 검증은 백엔드가 명령을 받을 때 합니다. 저는 권한 없는 사람에게 안 보여주는 것까지만 합니다.

왜 로그인 때 한 번인가 역할은 세션 중에 거의 안 바뀝니다. 실제 방어선이 백엔드라 화면이 조금 늦게 알아도 위험하지 않고요.

VZ-C-02 · 외부 소스 부재 시 동작

한 줄로 말하면 > “다른 파트가 아직 준비 안 됐어도 가시화는 혼자 돌아가야 합니다.”

쉽게 풀면 전송 규격도, 레지스트리 형태도, 액션 어휘집도, 영상 변환 지점도 아직 확정 전입니다. 그게 정해질 때까지 **손 놓고 기다리면 시연 준비가 늦어요**. 그래서 정적 파일과 대체 데이터로 개발을 진행합니다.

이건 방어적 요구사항입니다 — “다른 파트 진척에 우리가 막히지 않는다”를 요구사항으로 못 박아둔 것.

VZ-C-03 · 집약 계층 경계 표기

한 줄로 말하면 > “받은 값이 원본인지 요약인지, 요약이면 어디서 요약했는지를 확인하고 씁니다.”

자리만 열어두려던 게 이미 실사용이 됐습니다 “나중에 요약이 서버로 옮겨가면”이라고 썼는데, 이미 그렇게 설계돼 있습니다. 평소 지표는 구역 단위로 요약된 값이 올라와요.

쉽게 풀면 이미 요약된 값을 원본인 줄 알고 **또 평균 내면** 완전히 틀린 숫자가 나옵니다. 백엔드가 값마다 “원본인가 요약인가, 요약이면 어느 계층에서”를 표시해 보내주니, 저는 그걸 확인하고 재집약을 피하면 됩니다.

왜 매번 확인하나 표시를 읽는 비용은 필드 하나입니다. 반면 재집약 오류는 **화면에 이상하게 보이지 않아** 발견이 늦어요. 숫자가 그냥 조금 다를 뿐이라서요.

VZ-C-04 · 권한 범위

한 줄로 말하면 > “‘무엇을 할 수 있나’와 함께 ‘어디까지’를 받습니다.”

이것도 실사용이 됐습니다 백엔드 역할 조회 API에 **범위(담당 구역 집합)**가 이미 들어 있습니다. 자리만 열어두려던 게 바로 쓸 수 있게 됐어요.

쉽게 풀면 같은 조작 권한이라도 **자기 담당 구역만** 만질 수 있어야 합니다. 담당이 아닌 구역까지 제어되면 책임소재가 성립하지 않아요. 화면은 범위 밖 대상의 제어 UI를 아예 안 보여줍니다.

왜 로그인 때 한 번인가 역할과 범위는 세션 중 거의 안 바뀝니다. 실제 방어선이 백엔드라 화면이 조금 늦게 알아도 위험하지 않고요.

VZ-C-05 · 설계 규모 가정 명시 (확인 요망)

한 줄로 말하면 > “이 설계가 몇 개까지 감당하는지를 숫자로 적어줘야 합니다.”

쉽게 풀면 지금 제 주기 숫자는 전부 “구역 하나에 장치 수십 개”를 전제로 나왔습니다. 그런데 그 전제가 문서 어디에도 없어요.

그래서 “이거 국가 규모도 되나요?” 같은 질문이 나오면 답할 근거가 없습니다. 시연 규모와 설계 상한을 숫자로 못 박아두면 문서로 답할 수 있게 됩니다.

이건 자리 확보가 아니라 확인 요망입니다. 숫자는 팀이 합의할 사항이라 제가 정할 수 없어요.

VZ-C-06 · 원천 종류 표기 (확인 요망)

한 줄로 말하면 > “이 값이 진짜 장치에서 온 건지 시뮬레이션에서 온 건지를 표시와 함께 받습니다.”

왜 지금 나왔나 트윈 화면을 Unity로 만들면서 **Unity가 시뮬레이터를 겸하게** 됐습니다. 그러면 가짜 로봇 상태가 진짜와 똑같은 봉투로 올라와요. 지금 계약에는 이 둘을 가를 필드가 없습니다.

쉽게 풀면 화면에서 구분이 안 되는 것도 문제지만, 더 큰 건 기록입니다. 제어 명령을 보낼 때 “그때 시뮬레이션 중이었다”가 감사에 안 남으면, 나중에 사고를 조사할 때 훈련으로 연 수문과 실제로 연 수문이 같은 모양으로 남습니다.

그래서 값을 받을 때 표시를 읽어 화면에서 구분하고, 명령을 보낼 때도 지금 어느 원천을 보고 있었는지를 함께 실어 보냅니다.

왜 지금 자리를 여나 VZ-C-03(집약 계층 표기)과 똑같은 모양입니다. 지금은 필드 하나지만, 나중에 끼우려면 **발행·저장·감사·화면 전 경로**를 다 손대야 해요. 캡스톤에서는 시뮬레이터가 하나뿐이라 티가 안 나지만, 국가 인프라 규모에서는 **훈련과 실제 상황을 기록으로 구분하지 못하는** 문제가 됩니다.

이것도 확인 요망입니다. 표시를 붙여 보내는 건 하드웨어와 백엔드라서, 제가 정할 수 있는 건 “읽어서 구분해 보여준다”까지입니다.

디버깅 (VZ-D) — 8개 (2026-08-27 신설)

8/26 회의에서 논문 주제가 “디버깅을 위한 가시화”로 확정되면서 생긴 계열입니다. 앞의 33개가 “지금 무엇이 어떤 상태인가”를 그린다면, 이 8개는 “왜 실패했는가”를 되짚습니다. 다루는 것이 최신값이 아니라 **쌓인 기록**이라는 점이 다릅니다.

VZ-D-01 · 3계층 임무 모델 표시

한 줄로 말하면 > “임무 하나를 마일스톤 → 태스크 → 액션 아이템 세 계층으로 펼쳐 보여줍니다.”

쉽게 풀면 “415호에서 503호로 가라”는 명령을 이렇게 쪼갭니다.

계층

마일스톤

태스크

액션 아이템

태스크는 일렬이 아니라 **갈래가 갈라지고 다시 합쳐지는 그래프**입니다. 그리고 **평가를 통과해야 태스크가 끝납니다** — 실행이 끝난 것과 평가가 끝난 것은 다릅니다.

지금 걸린 것 □ **이 계층을 만들어 줄 파트가 없습니다.** 계획 생성·검증(AI-D-01·02·04)이 삭제됐습니다.

VZ-D-02 · 실행 추적 기록 수신·보관

한 줄로 말하면 > “무슨 일이 언제 일어났는지를 한 줄씩 계속 덧붙이기만 하고, 지우지 않습니다.”

쉽게 풀면 기록 한 줄에는 **순번·시각·어느 계층의 무슨 노드·무슨 일·그 일을 한 주체가 AI인지 백엔드인지 사람인지**가 들어갑니다. 마지막 항목이 핵심입니다 — **사람이 중간에 끼어들어 값을 고친 지점이 기록에 없으면 나중에 되짚을 때 도구가 거짓말을 합니다.**

왜 이 주기인가 사건에 종속되어 주기가 없습니다. 다만 로봇 상태 50 ms, 액추에이터 50 ms로 올라오므로 초당 수십 건이 될 수 있어, **수신은 전량 하되 화면 반영만 100 ms 창으로 병합**합니다. 기록을 요약해 받으면 되감기와 경로 격리가 성립하지 않아 수신 자체는 줄이지 않습니다.

안 하면 어떻게 되나 실패했을 때 화면에 남는 건 “실패함” 한 줄뿐입니다. 되짚을 것이 없어요.

VZ-D-03 · 계층 경계 강제

한 줄로 말하면 > “마일스톤·태스크에 모터니 바퀴니 하는 말이 못 들어가게 **검사로 막습니다.**”

쉽게 풀면 회의에서 “태스크까지는 범용, 액션 아이템부터 기종 의존”으로 합의했는데, **합의만으로는 무너집니다.** 편의상 태스크에 기종 필드가 하나씩 붙기 시작하면, 그 순간 같은 임무를 다른 로봇에 재사용할 수 없게 됩니다.

그래서 계약 스키마에서 막고, **새 로봇 프로파일을 추가할 때 상위 두 계층 코드가 한 줄도 안 바뀌는지를** 검증 스크립트로 확인합니다.

VZ-D-04 · 시점 복원(되감기)과 임무 이력

한 줄로 말하면 > “슬라이더를 끌면 그 시점의 화면이 그대로 복원됩니다. 지난 임무도 골라서 되감습니다.”

쉽게 풀면 기록이 남아 있으니 앞에서부터 점으면 임의 시점 상태가 나옵니다. 실패·재실행이 있던 지점으로 **바로 점프**할 수 있어야 합니다.

중요한 원칙 되감기 중에는 명령을 차단합니다. 과거 시점을 보고 있는 화면에서 낸 명령이 현재 상태에 적용되면 의도하지 않은 대상에 나갑니다.

별도 화면이 아닙니다. VZ-D-01 화면의 재생 모드입니다.

VZ-D-05 · 실패 경로 격리

한 줄로 말하면 > “실패한 노드에서 거꾸로 따라가 관련된 것만 남기고 나머지를 접습니다.”

쉽게 풀면 7개 중 3개만 남으면 사람이 볼 것이 절반 이하로 줍니다. 회의록의 “**실패한 경로만 딱 띄워서 추적**”이 이겁니다.

놓치면 안 되는 것 좁혔는데 정답을 잘라내면 그건 실패입니다. 얼마나 줄었는지와 함께 근본 원인이 남은 집합 안에 있는지를 같이 검사합니다.

VZ-D-06 · 실행 상태 8종 표시

한 줄로 말하면 > “특히 ‘안 돌렸다’(건너뛴)와 ‘못 돌았다’(미수행)를 구분합니다.”

쉽게 풀면 대기 · 진행 · 완료 · 실패 · 건너뛴 · 평가 대기 · 미수행 · 재실행(회차 표시) 여덟 개입니다.

건너뛴은 조건상 일부러 넘긴 것이고, 미수행은 **앞이 실패해서 실행 자체가 일어나지 않은 것**입니다. 디버깅에서 이 둘은 완전히 다른 단서예요 — 하나로 합치면 원인을 위에서 찾을지 아래에서 찾을지 판단할 근거를 잃습니다.

색만으로 여덟 개를 구분하지 않습니다. 테두리 모양·아이콘·회차 표기를 함께 씁니다.

VZ-D-07 · 대상 배정과 대상 상태 조회

한 줄로 말하면 > “하드웨어 목록에서 **끌어다 놓아** 마일스톤에 배정하고, 그 대상의 상태를 노드와 상세에서 조회합니다.”

쉽게 풀면 배정이 마일스톤과 액션 아이템을 잇는 고리입니다. 배정 결과가 액션 아이템의 대상과 어댑터가 돼요.

노드에는 연결 상태와 전원 정도를, 액션 아이템 상세에서는 배터리·링크 품질·주소·펌웨어까지 봅니다. 디버깅에서 “**코드가 틀렸나 장비가 문제인가**”를 가르는 첫 질문이 여기서 끝나야 합니다.

실행 전에 드러나야 하는 것 배정된 대상이 오프라인이면 그 태스크는 실행하면 반드시 실패합니다. 그게 실패 이후에야 보이면 이 도구는 사후 추적으로만 쓰입니다.

VZ-D-08 · 화면에서의 실행 개입

한 줄로 말하면 > “정지·재시작·중단, 그리고 실패한 값을 고쳐 다시 돌리는 것까지 화면에서 합니다.”

쉽게 풀면 보기만 하는 도구가 아니라 **양방향 도구**입니다. 임무 정지·재시작·중단, 파라미터 수정 후 재실행, 액션 아이템 단독 실행, 결함 일부러 주입하기, 그리고 **실제로 움직이지 않고 결과만 확인하기**.

정지는 액션 아이템 경계에서만 멈춥니다. 물리 동작 중간에 끊으면 로봇이 어중간한 자세로 남고, 그 상태는 기록으로 설명되지 않습니다.

타협하지 않는 것 여기서 나가는 조작은 전부 기록에 남습니다. 예외 없습니다. 사람의 개입이 빠지면 되감기와 경로 격리가 **잘못된 결론**을 냅니다 — 이걸 성능 문제가 아니라 도구의 신뢰성 문제입니다.

생성 (VZ-G) — 5개 (2026-08-27 신설 · 인수분)

원래 AI 파트가 하기로 했던 일입니다. AI-D-01(서브태스크 생성)·AI-D-02(검증)가 엑셀 신판에서 삭제됐고, AI 팀원이 논문 작업을 병행하게 되어 **가시화가 가져왔습니다**.

이 계열이 생기면서 이 문서의 전제 하나가 깨집니다. 원래 가시화는 “받아서 그리는” 레이어였는데, 이제 **임무 계층은 직접 만듭니다**. 관측값(센서·영상·상태)은 여전히 만들지 않습니다.

발화 텍스트

- VZ-G-01 마일스톤으로 쪼개기 ← 이게 곧 '의도 해석'입니다
- VZ-G-02 태스크 생성 (DAG)
- VZ-G-03 검증
- VZ-G-04 액션 아이템 전개 (어댑터)
- 실행
- VZ-G-05 평가 판정 ← 통과해야 태스크가 끝납니다

VZ-G-01 · 발화 텍스트 → 마일스톤 분리

한 줄로 말하면 > “415호에서 503호로 가라’를 마일스톤 목록으로 쪼갭니다.”

쉽게 풀면 STT가 만든 텍스트를 받아 임무 하나와 마일스톤 목록으로 나눕니다. 마일스톤은 **완전히 추상**이라 로봇 종류가 안 나옵니다 — 415호 → 복도 → 엘리베이터 → 5층 → 503호.

결과는 **제안 상태**로 뜨고 사람이 수락해야 아래로 넘어갑니다.

여기가 8/25부터 비어 있던 구멍입니다 “명령 의도를 해석하는 파트가 없다”가 계속 걸려 있었는데, 이 단계가 곧 **의도 해석**입니다. 해석기가 두 파트에 나뉘지 않으니 같은 발화가 서로 다른 결과를 내는 문제도 없어졌어요.

왜 별도 요구사항인가 마일스톤을 잘못 쪼개면 그 아래가 전부 무너집니다. **분리 자체가 실패 지점**이라 생성 근거를 기록에 남겨야 역추적이 맨 위까지 닿습니다.

VZ-G-02 · 마일스톤 → 태스크 생성

한 줄로 말하면 > “마일스톤마다 실제로 할 수 있는 단위로 쪼갭니다. 일렬이 아니라 그래프로요.”

쉽게 풀면 태스크는 갈래가 갈라지고 다시 합쳐집니다. 회의록의 “*flow → process — 여러 갈래가 다시 하나의 박스로 모이는 형태*”가 이겁니다. 일렬 번호로 두면 분기·합류가 사라지고, 그러면 실패 경로 역추적(VZ-D-05)이 성립하지 않습니다.

태스크까지가 범용입니다. 여기에 로봇 기종이 나오면 안 됩니다.

VZ-G-03 · 태스크 검증

한 줄로 말하면 > “만든 걸 실행 전에 검사합니다. 만드는 쪽과 검사하는 쪽을 나눠서요.”

쉽게 풀면 구조·순서·의존 관계에 순환이 있는지·사전조건·자원·배정된 대상이 할 수 있는 동작 인지를 봅니다. 확인 못 하는 조건이 있으면 그 태스크만 실행 불가로 표시하고 뭐가 부족한지 적습니다.

왜 모듈을 나누나 같은 사람이 만들더라도 나눕니다. 생성기가 스스로를 검증하면 같은 오해를 두 번 반복할 뿐이고, 재현 가능한 검증이라야 실패했을 때 생성 탓인지 검증 탓인지 가를 수 있어요.

VZ-G-04 · 태스크 → 액션 아이템 전개

한 줄로 말하면 > “검증 통과한 태스크를 그 로봇이 알아듣는 실제 명령값으로 바꿉니다.”

쉽게 풀면 “이동”이 Go1에서는 “3.4 m를 초속 0.6 m로 직진”이 되고, 바퀴형에서는 다른 값이 됩니다. 바뀌는 건 이 단계뿐이고 위의 두 계층은 그대로입니다. 기종별 규칙은 어댑터 프로파일 파일에만 둡니다.

왜 실행 직전이 아니라 검증 직후인가 전개 결과를 승인 화면에서 미리 봐야 하고(VZ-U-07), 전개하다 드러나는 불가능 (그 로봇이 지원하지 않는 동작)을 실행 전에 잡아야 하기 때문입니다.

VZ-G-05 · 평가 기준 생성 및 판정

한 줄로 말하면 > “태스크마다 ‘이러면 성공’을 정해 두고, 끝났을 때 그 기준으로 판정합니다.”

쉽게 풀면 회의에서 “평가가 끝나야 태스크가 종료된다”로 합의한 부분입니다. 실행이 끝난 것과 태스크가 끝난 것은 다릅니다 — 도착 오차 0.2 m 이내인지 같은 걸 봐야 끝납니다.

타협하지 않는 것 판정에 필요한 값이 안 왔으면 시간이 지났다는 이유로 통과시키지 않습니다. 그러면 “평가가 끝나야 끝난다”는 합의가 무의미해져요. 규칙으로 못 정하는 건 사람 판정으로 넘깁니다.

지금 걸려 있는 것

다른 파트에 답이 걸려 있어 진행이 반쯤 묶인 항목입니다. **2026-08-25 기준**으로 다시 정리했습니다.

풀린 것

무엇	어떻게 풀렸나
좌표 기준·출처 필드	백엔드 단독 변환으로 확정
프레임 참조 형식 통일	공통 규약으로 확정
명령 상관키	백엔드 발급으로 확정
레지스터라 조화 형태	조회 API 제공 확정
요구사항이 「브라우저」에 묶여 있던 것	백엔드가 먼저 「 가시화 클라이언트 」로 일 반화했습니다(BE-T-03). Unity 뷰어가 계 약상 정당한 클라이언트가 됐어요
재접속 캐시 범위	제가 답할 차례였고 채널 단위로 갈라 회신했 습니다 (VZ-I-02 참고). 팀 전달만 남았습니 다

남은 것

무엇	상태	정하지 않으면
영상을 뷰어로 내보내는 지점	□ 자리는 생겼고 주인이 없음	실제 카메라를 붙이는 순간 막힙니다
계획 생성·검증의 담당 파트	□ 가시화가 인수 (VZ- G-01~05)	—
명령 의도 해석의 주체	□ VZ-G-01이 곧 의도 해석	8/25부터 걸려 있던 것이 함 께 풀렸습니다
재생성 판단	□ 확장점 E9로 분류	미결이 아니라 나중에 볼 것 으로 정리했습니다 (아래 참 고)
로컬 LLM 사양·자원	쉴 필요	생성이 LLM에 걸리는데 어 느 모델을 어디서 돌릴지가 없습니다. 폐쇄망이 전제라 외부 API는 기본 경로가 못 됩니다
미확인 후보를 묶어 주는 주 체	□ 8/27에 사라짐	AI-L-02 삭제. 확인 요청이 건마다 와서 사용자 부담이 커집니다
관측 export 60초 vs 요 약 pull 15초	약 쪽에 맞출지 합의 대기	하드웨어 지표가 15초 질의 에도 같은 점을 네 번 반복합 니다
오프라인 표시의 약 4초 지 연	표 방식 합의 대기	감추면 “방금까지 정상이었 다”는 오해를 만듭니다

“이 배치엔 그 기능이 없다”를 알려주는 경로	없음	미배포와 장애를 화면이 구분 못 합니다
원천 종류 표기	불일치	주체와 형식이 합의 대 시뮬레이션 조작과 실물 조작 이 감사에서 구분 안 됩니다
설계 규모 가정	췌하는	팀 합의 대기 모든 주기 산정의 전제가 문 서에 없습니다
좌표계 손잡이 규약	확인 필요	Unity는 왼손 좌표계라 좌우 가 뒤집힙니다

영상 — “담당자가 없다”에서 두 걸음 나아갔습니다

전에는 “카메라 원본을 뷰어 형식으로 바꿀 사람이 세 파트 통틀어 없다” 였습니다. 지금은 이렇게 좁혀졌어요.

구간

카메라 → 엡지

원본 규격 흡수

미디어가 별도 평면

미디어 → 뷰어

그래서 질문이 “누가 만드나” 가 아니라 “미디어 경로에 뷰어용 출력 분기를 다느냐, 그 어댑터는 누구 것이냐” 가 됐습니다. 도구(GStreamer)가 이미 시스템 안에 있으니 WebRTC·HLS 내보내기는 자연스러운 확장입니다.

계획 생성 — 생겼다가 같은 날 메워졌습니다

AI-D-01·02·04가 사라져서 **마일스톤·태스크를 만들 파트가 아무도 없는** 상태였습니다. 8/26 회의에서 확정한 3계층 모델이 바로 이 요구사항이 만들어 주던 것이라, 도구를 아무리 잘 만들어도 채울 데이터가 없으면 목 데이터로만 도는 도구가 될 뻔했어요.

가시화가 인수하는 것으로 정리됐습니다 (VZ-G-01~05). AI 팀원이 논문 작업을 병행하게 된 사정도 있었습니다. 부수적으로 8/25부터 걸려 있던 **의도 해석 구명도 같이 메워졌습니다**.

다만 인수하면서 새로 생각할 것이 둘 생겼습니다.

- **생성기와 디버거가 한 레이어에 있게 됐습니다.** “디버깅 도구가 좋다”와 “생성기가 좋다”가 섞이지 않게 해야 합니다. 결함 주입 실험은 **근본 원인을 아는 합성 데이터**를 쓰니 괜찮지만, 사용자 실험에서는 두 변인을 갈라 설계해야 합니다.
- **일정이 늘었습니다.** 다만 1단계(9/7 HCI 전달본)는 그대로 목 데이터로 갑니다. 생성기는 2단계에 넣습니다 — 안 그러면 전달이 밀립니다.

재생성 판단은 확장점(E9)으로 뺐습니다

계획을 실행하다 **전제가 깨졌을 때 남은 부분을 다시 만드는** 판단입니다. 엘리베이터가 점검 모드로 바뀌면 T-34 이후가 전부 무의미해지는데, 그때 계단 경로로 갈아탈지를 정하는 일이에요. 구 AI-D-04가 하던 것입니다.

본체에는 넣지 않기로 했습니다. 필수 동작이 아니라 확장 기능이라서요.

무엇을 다시 하나

그래프

누가 결정

뜻

지금은 전제가 깨지면 **사람이 임무를 중단하고 다시 발화**합니다. 그 경로가 이미 있어요(VZ-D-08).

빠는 이유가 하나 더 있습니다. 이 판단은 “**실제로 깨졌을 때만**”을 가르는 게 핵심인데, 과민하게 만들면 계획이 계속 다시 만들어져 로봇이 앞으로 못 나갑니다. 그리고 그 재생성 사건들이 트레이스를 덮어서 — **디버깅하려고 만든 기록이 노이즈로 가득 찹니다**. 우리 도구 목적과 정면으로 부딪쳐요.

계약은 이미 열려 있습니다. derived_from과 kind=derived가 그 자리라, 나중에 붙여도 스키마가 안 바뀝니다. 본체를 다 만들고 확장을 검토할 때 다시 봅니다.

(이전) 계획 생성이 비어 있던 상황

이번에는 뜻이 바뀐 게 아니라 **항목 자체가 사라졌습니다**. AI-D-01(서브태스크 생성)·AI-D-02(검증)·AI-D-04(재생성 판단) 세 개가 엑셀 신판에서 없어졌고, **다른 파트 시트 어디에도 대체가 없습니다**.

이게 왜 제일 급하냐면, 8/26 회의에서 확정된 3계층 모델(마일스톤 → 태스크 → 액션 아이템)이 **바로 이 사라진 요구사항이 만들어 주던 것이기** 때문입니다. 화면과 도구를 아무리 잘 만들어도 **채울 데이터를 만드는 파트가 없으면** 목 데이터로만 도는 도구가 됩니다.

가능한 답은 셋입니다 — AI 파트가 다시 가져가거나, 백엔드가 가져가거나, 가시화가 로컬 LLM(VZ-L-04)으로 직접 만들거나. 셋 다 각자 파트의 범위가 달라지는 결정이라 **혼자 정할 수 없습니다**.

의도 해석 — 8/25에 생긴 구멍입니다

번호(AI-D-03)는 살아 있는데 **뜻이 바뀌어서** 생긴 일입니다. 자동 검사로는 안 잡힙니다. 이 건은 “**ID를 재사용하지 말자**”는 안건의 실제 사례가 됐습니다 — 가설이 아니라 사고가 한 번 난 겁니다.

기능 유무를 알려주는 경로 — 새 상태 두 개가 필요할 수 있습니다

지금 화면의 상태 표시는 **정상 / 장애 / 미배포 / 판단 불가** 넷입니다. 그런데 답을 자리가 없는 것이 둘 있어요.

- 이 배치에는 그 기능이 **아예 없다** (선택 기능 미배포)
- 성능 저하 모드로 **돌고 있다**

장애와 미설치를 섞으면 관제사가 고칠 수 없는 것을 고치려 합니다. 다만 이건 신설 후보이지 확정이 아닙니다 — 그 상태가 화면까지 오는 경로가 없으면, 요구사항을 만들어도 읽을 값이 없습니다.

자주 나올 질문과 답

Q. 왜 다 백엔드를 거치나요? 직접 붙으면 빠를 텐데. > 뷰어는 사용자 손 안에서 돕니다. 접속 비밀번호를 뷰어에 내려주면 누구나 볼 수 있어요. 그리고 권한 제한이나 요청량 제한을 걸 곳도 백엔드밖에 없습니다.

Q. 상태를 왜 굳이 세 개로 나눠 받나요? 하나면 편한데. > “연결은 됐는데 기기가 고장난 상태”를 하나로 표현할 수 없습니다. 아는 주체가 셋이라(기기·서버·배포 시스템) 층도 셋이어야 합니다.

Q. 감사는 가시화가 쓰는 게 자연스럽지 않나요? > 그러면 “누가 했다”를 클라이언트가 자기 입으로 말하는 구조가 됩니다. 조작할 수 있어요. 권한은 백엔드가 지키는데 기록만 뷰어가 쓰면 앞뒤가 안 맞습니다.

Q. 계획 승인을 화면에 두면 느려지지 않나요? > 느려지는 게 목적입니다. 승인 없이 자동 실행하면 사고가 났을 때 책임소재가 성립하지 않아요. 대신 근거를 잘 보여줘서 **승인 판단 자체는 빠르게** 만드는 게 제 몫입니다.

Q. 주기 숫자는 어떻게 정했나요? > 대부분 다른 파트가 정한 주기에서 따왔습니다. 원천이 15초에 한 번 주는 걸 1초마다 물어봐야 같은 값만 옵니다.

Q. 지금 못 하고 있는 게 있나요? > 33개 중 22개는 돌아갑니다. 위 “지금 걸려 있는 것” 표대로 여섯 가지가 다른 파트에 걸려 있는데, 성격이 둘로 갈립니다 — **영상 출력 분기와 의도 해석**은 상대 파트가 담당을 정해야 하는 것이고, 나머지 넷은 형식·숫자 확인이면 됩니다.

Q. 상대 요구사항의 번호가 그대로면 안심해도 되나요? > 아니요. 이번에 번호는 살아 있는데 뜻이 바뀐 경우를 겪었습니다(AI-D-03). 참조가 끊기면 검사로 잡히지만 뜻만 바뀌면 안 잡힙니다. 그래서 시트가 갱신될 때마다 **제 요구사항이 가리키는 상대 항목의 이름까지 전수** 대조하는 것을 습관으로 두고 있습니다.